



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

DecDEC: A Systems Approach to Advancing Low-Bit LLM Quantization

Yeonhong Park, Jake Hyun, Hojoon Kim, and Jae W. Lee, *Seoul National University*

<https://www.usenix.org/conference/osdi25/presentation/park-yeonhong>

This paper is included in the Proceedings of the 19th USENIX Symposium
on Operating Systems Design and Implementation.

July 7–9, 2025 • Boston, MA, USA

ISBN 978-1-939133-47-2

Open access to the Proceedings of the 19th USENIX Symposium
on Operating Systems Design and Implementation is sponsored by



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

DecDEC: A Systems Approach to Advancing Low-Bit LLM Quantization

Yeonhong Park* Jake Hyun* Hojoon Kim Jae W. Lee

Seoul National University

Abstract

Quantization of Large Language Models (LLMs) has recently gained popularity, particularly for on-device settings with limited hardware resources. While efficient, quantization inevitably degrades model quality, especially in aggressive low-bit settings such as 3-bit and 4-bit precision. In this paper, we propose DecDEC, an inference scheme that improves the quality of low-bit LLMs while preserving the key benefits of quantization: GPU memory savings and latency reduction. DecDEC stores the residual matrix—the difference between full-precision and quantized weights—in CPU, and dynamically fetches the residuals for only a small portion of the weights. This portion corresponds to the salient channels, marked by activation outliers, with the fetched residuals helping to correct quantization errors in these channels. Salient channels are identified *dynamically* at each decoding step by analyzing the input activations—this enables adaptation to the dynamic nature of activation distribution, thus maximizing the effectiveness of error compensation. We demonstrate the effectiveness of DecDEC by augmenting state-of-the-art quantization methods. For example, DecDEC reduces the perplexity of a 3-bit Llama-3-8B-Instruct model from 10.15 to 9.12—outperforming its 3.5-bit counterpart—while adding less than 0.0003% to GPU memory usage and incurring only a 1.7% inference slowdown on NVIDIA RTX 4050 Mobile.

1 Introduction

Recent advancements in Large Language Models (LLMs) based on the Transformer architecture [59] have shown great potential to reshape our daily lives [7, 40, 44, 55, 56]. However, their deployment costs pose a significant challenge, as large model sizes increase memory requirements and latency, limiting their use cases [60]. Quantization is a promising solution for reducing the LLM deployment costs [28, 69]. By lowering the model precision, quantization addresses both memory

limitations and inference latency. The importance of quantization is pronounced for on-device deployments, where strict memory budgets often make model compression mandatory rather than optional. These scenarios typically require careful tuning of quantization levels to achieve an optimal balance between model size and quality [8, 14, 15, 58, 61].

While quantization may allow gigantic LLMs to fit into small-memory devices, it often leads to model quality degradation due to the inevitable loss of information. This is especially true for low-bit settings, such as 3-bit and 4-bit quantization, which are often used to accommodate the parameter sizes of LLMs [19, 31, 36]. This raises a key research question that this paper addresses: given a quantized LLM configured with the best possible effort under the memory budget, is there a way to recover the quality loss caused by quantization?

Leveraging external memory offers a potential solution to this problem. Specifically, on heterogeneous computing platforms where the CPU and GPU are connected via a PCIe interconnect—a common architecture in desktops and laptops—CPU memory becomes a viable option. Additional information that may be used to mitigate quantization errors can be stored in CPU memory and fetched at runtime, avoiding any additional GPU memory overhead.

However, utilizing CPU memory for GPU inference presents a critical challenge: the slow data transfer between the CPU and GPU can create a bottleneck for inference latency. To mitigate this issue, the volume of data transferred must be carefully controlled. Therefore, designing a system that effectively utilizes CPU memory requires the identification of the minimal set of additional information that can substantially enhance the quality of quantized models, while keeping the impact on latency minimal.

A clue for this problem comes from the well-known fact that not all channels in the weight matrix are equally important for quantization. Some channels, referred to as *salient channels*, are more critical, primarily due to the presence of activation outliers [6, 10, 25]. When certain input activation values are particularly large, the quantization errors in the corresponding weight channels—those multiplied by these large

*Equal contribution

activation values—are amplified, making the channels salient. By identifying these channels and selectively fetching error compensation terms from CPU memory, we can maximize the quality boost while minimizing data transfer.

Hence, the identification of the salient channels is essential for overcoming the bandwidth limitation, allowing for the transfer of only impactful information. Previous works that attempt to improve quantization algorithms by addressing activation outliers analyze the activation value distribution on a calibration data to predetermine the salient channels [10,27,33,36]. This approach is suboptimal as it statically designates certain channels as salient throughout the inference, while the real distribution of activation values—and thus the distribution of salient channels—changes dynamically at each decoding step. Accounting for this dynamic nature is crucial for the accurate identification of salient channels.

Thus, in this paper, we introduce DecDEC (**Dec**oding with **D**ynamic **E**rror **C**ompensation), an inference scheme for quantized LLMs that dynamically identifies salient channels and compensates for quantization errors in these channels, in real time. DecDEC enhances model quality while fully preserving the two main benefits of quantization: GPU memory savings and latency reduction. To achieve this, DecDEC stores the residuals of the quantized weight matrices in CPU memory, fetching only the parts that correspond to the dynamically identified salient channels for error compensation. Although small in size, these residuals provide a significant quality boost. This dynamic error compensation is performed concurrently with inference by an optimized GPU kernel, ensuring that all additional operations are seamlessly integrated into the existing workflow, minimizing inference slowdown. Below are the key contributions of our work:

- We provide an in-depth analysis on the dynamic nature of activation outliers in LLM inference.
- We present DecDEC, an inference scheme that enhances quantized LLMs by dynamically identifying salient channels and compensating quantization errors in them.
- We introduce a tuner for DecDEC that recommends system parameters to satisfy a target latency bound.
- We evaluate DecDEC across five different consumer-grade GPUs, demonstrating significant quality improvement with minimal memory and latency overhead.

2 Background

2.1 LLM Inference

Figure 1 presents a description of the modern LLM architecture and its inference flow. LLMs consist of multiple Transformer decoder blocks [59], each containing multiple linear layers, which account for most of the inference time, alongside other components like self-attention and normalization.

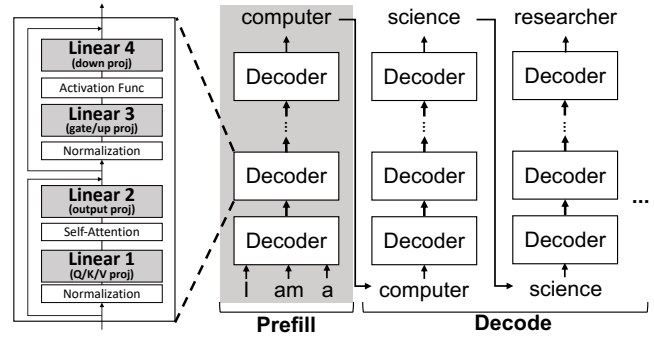


Figure 1: LLM inference.

LLM inference involves two phases: prefill and decode. In the prefill phase, all input tokens (e.g., 'I', 'am', 'a') are processed in parallel to generate a single output token (e.g., 'computer'). The decode phase begins subsequently, where the output token of the previous step is fed back into the model to generate the next token, repeated until the end of the sequence. This sequential nature of the decode phase makes it the primary latency bottleneck.

The decode phase is particularly memory-bound, as only one token is processed at a time, reducing the linear layers to GEMV operations. In data center settings, this issue can be alleviated by batching together multiple queries [32,66]. However, this usually cannot be applied to on-device inference, where LLMs serve only individual users.

2.2 LLM Quantization

Quantization for LLMs—a popular compression technique that reduces both memory usage and inference latency—can be categorized into two main types [28,69]: weight-activation quantization [10,34,37,49,64,65] and weight-only quantization [9,11,12,17,19,30,31,33,36,57]. Each are suited to different inference scenarios. Weight-activation quantization is primarily used in datacenter settings, where both memory and computational costs must be minimized to improve throughput. Quantizing both weights and activations allows for the efficient use of low-precision arithmetic units (e.g., INT4, INT8, FP8) available on modern GPUs [1]. In contrast, for on-device inference, weight-only quantization is the preferred approach [31,69]. In this approach, quantized weights are loaded from memory and dequantized on-the-fly to full precision (i.e., FP16), before being multiplied with the full-precision activations [23]. Though it only reduces memory traffic, this is sufficient to speed up on-device inference where memory is the bottleneck, as discussed in Section 2.1.

Weight-only quantization can be further divided into two sub-categories: quantization-aware training (QAT) [11,30,37], and post-training quantization (PTQ) [9,12,17,19,31,33,36,57]. While QAT yields better results by retraining to reduce quantization errors, its cost makes it impractical for many end-

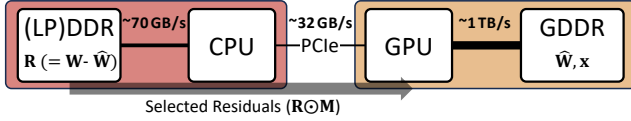


Figure 2: CPU-augmented inference for quantized LLMs.

users [31, 69]. As a result, PTQ—requiring no retraining—has become the preferred method for on-device LLM inference. Therefore, in this paper, we specifically focus on weight-only PTQ for on-device inference.

3 Augmenting Quantized LLM with CPU Memory

In this section, we propose leveraging CPU memory—an often underutilized resource in LLM inference, as GPUs are the de facto standard processors for this workload—as a means to augment quantized LLMs. Section 3.1 introduces the concept of CPU-augmented quantized LLMs while Section 3.2 and Section 3.3 explore its opportunities and challenges.

3.1 Concept

Goal. We aim to leverage CPU memory to improve quantized LLM quality without additional GPU memory costs. Quantization trades model quality for a smaller memory footprint. In constrained settings, practitioners optimize this trade-off within a fixed GPU memory budget by selecting a uniform bitwidth or applying fine-grained strategies such as layer-wise [8, 14, 15] or channel-wise [58, 61] allocation. Our goal is to then further improve quality post hoc by utilizing CPU memory, without increasing GPU memory usage.

Basic Mechanism. Figure 2 illustrates our concept of leveraging CPU memory to enhance quantized LLMs. We primarily target desktop or laptop platforms, where the GPU is connected to the CPU via a PCIe interconnect. As in conventional inference systems, the quantized weight parameters ($\hat{W}$) and activations (x) are kept in GPU memory. The difference here is that R —the residual between the original full-precision weights and the quantized weights—is stored in CPU memory. During the decode phase, residuals are fetched from the CPU to help compensate for quantization errors, potentially improving model quality. Due to the limited bandwidth of PCIe, which is typically an order of magnitude lower than GPU memory bandwidth (e.g., 32 GB/s vs. 1 TB/s), fetching the entire residual matrix would incur a prohibitive latency bottleneck. Therefore, only a small subset of residuals should be fetched in a selective manner. In short, this process augments each linear operation of quantized LLM from $\hat{W}x$ to $(\hat{W} + R \odot M)x$, where M is a binary mask that sparsifies R .

Key Research Question. A key research question in designing an effective CPU-augmented inference system for quan-

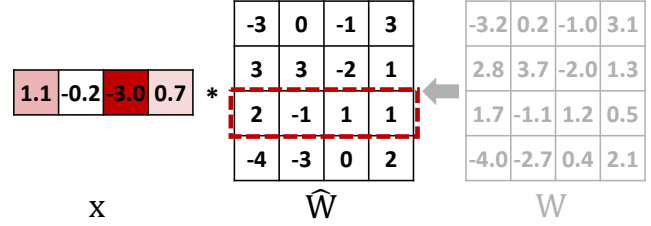


Figure 3: Activation outlier issue in weight quantization.

tized LLMs is determining a subset of residuals, or mask M . A good mask M should: 1) select portions of the residuals that contribute most to improving model quality within the bandwidth constraints, and 2) maintain a structured form that minimizes indexing overhead and ensures effective residual transfer, while enabling efficient processing on the GPU.

3.2 Opportunity: Not All Residuals Are Equally Important

Some residuals are more important than others—an opportunity that can be leveraged to determine M . This opportunity arises from the presence of activation outliers (i.e., activation values with large magnitudes), a well-known phenomenon in LLM inference [6, 10, 25, 27, 33, 36]. When certain activation values are noticeably large, even small quantization errors in the corresponding weight channels can be multiplied and amplified, leading to considerable perturbations in the output. Figure 3 illustrates this issue. In this example, the third input channel (third row) of the weight matrix is multiplied by the activation outlier, -3.0. We refer to such channels as *salient* channels. Constructing M at the input channel granularity based on the magnitude of input activations can satisfy two key conditions for an effective mask: selecting impactful portions of the residuals and maintaining a structured form.

Indeed, compensating for errors in salient channels using the corresponding residuals is highly effective. Figure 4 illustrates how the quantization error, defined as the mean squared error between the computation result with FP16 weights (Wx) and quantized weights ($\hat{W}x$), is reduced by sequentially replacing the input channels of the quantized weight with their corresponding FP16 values. For this analysis, we evaluate 3-bit and 4-bit versions of the LLaMA-3-8B-Instruct model [16], quantized using the state-of-the-art method AWQ [36], with a text sample from the C4 dataset [47] as the input prompt. All four linear layers in the 8th, 16th, and 24th decoder blocks are included in this evaluation. The quantization errors for both the 3-bit and 4-bit models drop rapidly when we progressively compensate for channels in descending order of their activation magnitudes (solid red and solid blue lines). This trend closely follows the activation magnitude distribution (solid black line), which represents the sorted activation magnitudes in descending order. In contrast, the reduction in

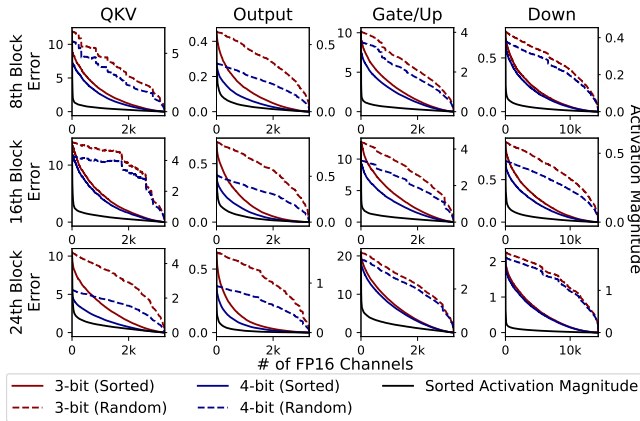


Figure 4: Quantization error reduction trends observed when replacing the input channels of quantized weights with FP16 values sequentially in sorted order (solid lines) and random order (dotted lines) for the 8th, 16th and 24th decoder block of Llama-3-8B-Instruct. The distribution of activation magnitudes in sorted order is also shown (black lines).

quantization error is significantly slower when input channels are compensated in random order, as shown by the dotted lines. This highlights the importance of prioritizing salient channels based on activation magnitude.

3.3 Challenge: Dynamic Nature of Activation Outliers

Identifying salient channels is challenging because the distribution of activation outliers changes dynamically by nature. While it is possible to infer salient channels by statically analyzing activation value statistics on a small calibration set [10, 27, 33, 36], such static approaches are suboptimal. Figure 5(a) shows the distribution of activation outliers—defined as activations with the top 5% magnitudes—in the down projection layer of the 8th, 16th, and 24th decoder blocks of the Llama-3-8B-Instruct model over 100 decoding steps, using a text sample for C4 dataset as the input prompt [47]. For visibility, only the first 512 channels are shown. While some channels (e.g., Channel 306 in 24th block, highlighted by an arrow) consistently exhibit high activation magnitudes and remain persistent outliers, the outlier distribution generally shows significant irregularity across decoding steps.

To quantify the dynamic nature of activation outliers, we calculate the recall rate of the top 1% and top 5% outliers identified through static analysis using a calibration set, compared to the true top 1% and top 5% outliers (ground truth) observed at each decoding step. We use a subset of the Pile dataset [21] for calibration, following prior work [36]. Specifically, we profile the average of the mean square of each activation value and use this as a metric for identifying outliers. Figure 5(b) presents the results, showing that the recall

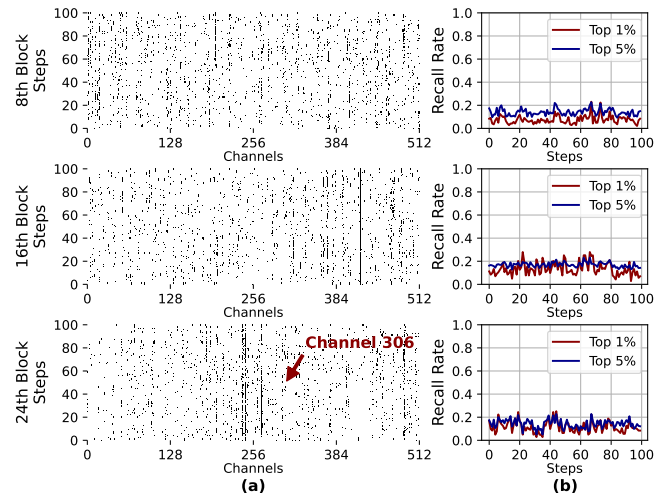


Figure 5: (a) Distribution of activation outliers (top 5%) and (b) Recall rate of static analysis-based outlier identification for the true top 1% and 5% outliers, across 100 decoding steps. The down projection layers in the 8th, 16th, and 24th decoder blocks of Llama-3-8B-Instruct model are used for profiling.

rate remains low ($\sim 20\%$) for both the top 1% and top 5% outliers. This highlights a clear limitation of static analysis, as it misses the majority of outliers at runtime, emphasizing the need for dynamic identification of salient channels.

4 DecDEC Design

4.1 Overview

In this section, building on the opportunity outlined in Section 3.2 and addressing the challenge described in Section 3.3, we propose DecDEC, a CPU-augmented inference system for quantized LLMs that performs decoding with dynamic error compensation. Figure 6 presents an overview of DecDEC. During the decode phase, DecDEC augments each linear layer, essentially a GEMV operation, with dynamic error compensation. To produce the final output $\mathbf{o}$, DecDEC adds an error compensation term $\mathbf{o}_{\text{dec}}$ to the base GEMV result $\mathbf{o}_b = \widehat{\mathbf{W}}\mathbf{x}$. $\mathbf{o}_{\text{dec}}$ is computed by multiplying the input vector with a subset of weight residuals selectively fetched from the CPU. This selection is performed *dynamically*, fully accounting for the variability of input vectors. To maximize the number of residual values fetched under PCIe bandwidth constraints, DecDEC stores and retrieves a quantized version of the residuals ($\widehat{\mathbf{R}}$), comprising the quantized values $Q_r(\mathbf{R})$ and associated metadata, instead of the full-precision residuals ($\mathbf{R}$). Here, Q_r is a quantizer that maps full precision residuals to low-bit form and differs from the base quantizer used for the weights, Q_b .

The dynamic error compensation process consists of four sequential steps. ① First, by investigating the input activation vector, DecDEC creates `sc_indices`, a list of salient channel

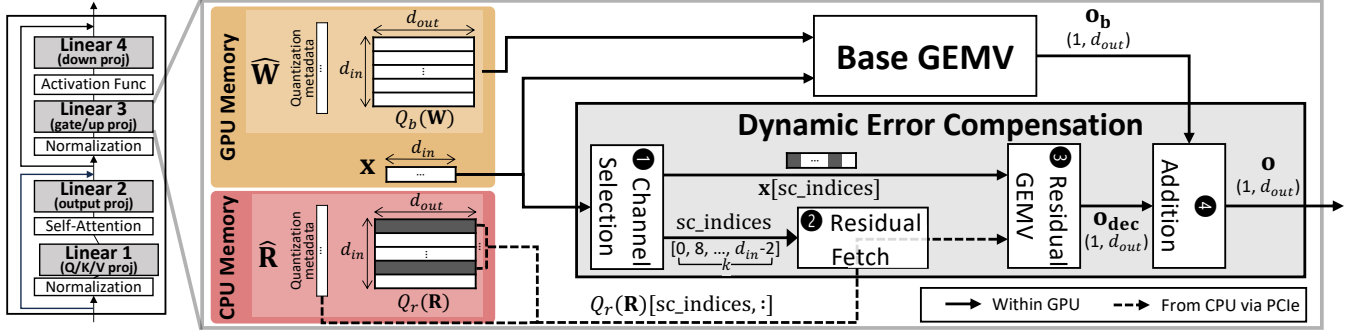


Figure 6: DecDEC overview.

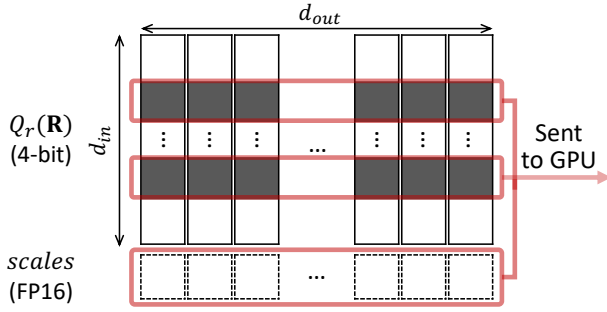


Figure 7: Quantization of weight residual.

indices. The number of salient channels to compensate, k , is a preconfigured parameter. This step is essentially a Top-K operation that selects the values in the input activation vector with the largest magnitudes. ② Next, a portion of the quantized residuals corresponding to the salient channels, $Q_r(\mathbf{R})[\text{sc_indices},:]$ (along with the necessary quantization metadata), is fetched from the CPU via PCIe. ③ The fetched residuals are then multiplied by the sparsified activation vector ($\mathbf{x}[\text{sc_indices}]$), producing $\mathbf{o}_{\text{dec}}$. ④ Finally, the resultant $\mathbf{o}_{\text{dec}}$ is added to the base GEMV result $\mathbf{o}_b$, producing the final output, $\mathbf{o}$. All steps run in parallel with the base GEMV on a different GPU stream, and must be highly efficient to remain hidden within base GEMV runtime.

The following sections provide details of each component of DecDEC. Section 4.2 explains how DecDEC performs the residual quantization. Section 4.3 details the GPU implementation of the dynamic error compensation. Section 4.4 illustrates how DecDEC configures system parameters, including k , the number of channels to compensate.

4.2 Residual Quantization

Figure 7 depicts the quantization scheme for the residuals. DecDEC employs 4-bit quantization for each output channel (i.e., column) of the residuals. To minimize metadata, symmetric uniform quantization is used. This approach requires

only a single scalar scale factor as metadata for each output channel. The residual quantizer for the i -th output channel ($Q_{r,i}$) is defined as:

$$Q_{r,i}(r) = \text{clip} \left(\text{round_to_int} \left(\frac{r}{S_i} \right), -7, 7 \right).$$

S_i , the scale factor, is determined through a grid search as the value that minimizes the mean squared error between the original and quantized weights.

Using this quantizer, each floating-point residual value r is projected to an integer between -7 and 7 . At runtime, the selected input channels of the quantized residuals (highlighted in Figure 7) and all the scale factors are fetched from the CPU. Each input channel of the quantized residuals, as well as the scales, are stored contiguously in CPU memory, enabling coalesced data transfers.

4.3 Efficient Implementation of Dynamic Error Compensation

The top priority in implementing dynamic error compensation is ensuring low latency, allowing its execution to remain hidden within the base GEMV execution time. To fetch a sufficient number of residual channels within this short time window, DecDEC introduces three key software optimization strategies: 1) zero-copy residual fetching, 2) fast approximate Top-K for channel selection, and 3) kernel fusion.

Zero-Copy Residual Fetch. DecDEC leverages CUDA zero-copy [42], instead of commonly used API functions such as `cudaMemcpy()` or `cudaMemcpyAsync()`, to fetch the residuals from CPU. These APIs rely on the direct memory access (DMA) engine for data transmission, which is efficient at transferring large data blocks but suboptimal for smaller transfers due to the DMA setup overheads. Fetching residuals, however, falls into the latter category. The granularity of residual fetching occurs at the row level of the quantized residual matrix. With 4-bit quantization and typical row lengths of a few thousand to tens of thousands, each data block transfer is only a few tens of KBs. For optimal PCIe bandwidth utilization, the data block size should ideally be at least a few

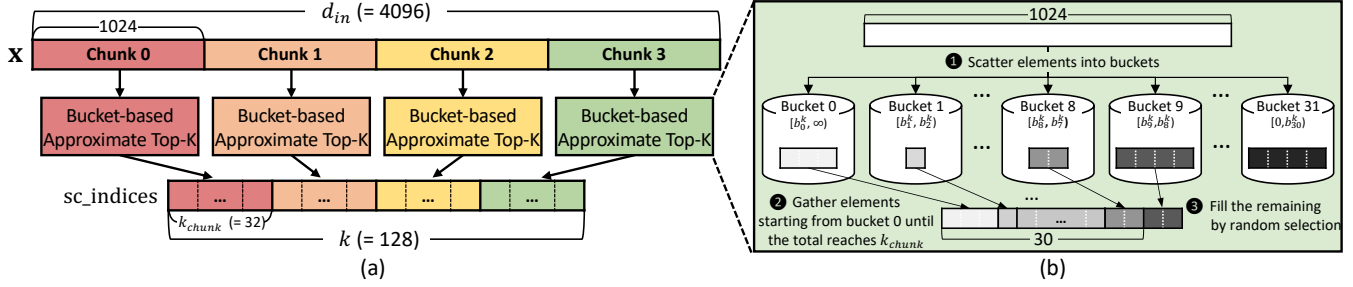


Figure 8: Fast approximate Top-K operation of DecDEC.

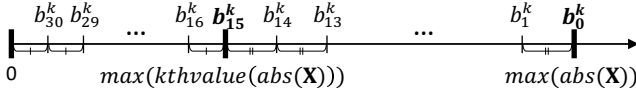


Figure 9: DecDEC's approximate Top-K bucket boundaries.

hundred KBs (e.g., 256 KB) [41, 46]. In zero-copy access, the GPU directly sends cacheline-sized memory requests, making it suitable for fine-grained data access. While zero-copy access has the disadvantage of occupying GPU cores to generate memory requests—potentially slowing down other concurrently running kernels—this is not a major issue for DecDEC. The concurrent kernel for our case, the base GEMV, is typically memory-bound, so using fewer cores for this kernel is unlikely to have significant impact on its execution time.

Fast Approximate Top-K for Channel Selection. For channel selection, instead of an exact Top-K operation, DecDEC employs an approximate Top-K method that is fast and GPU-friendly while maintaining precision. Figure 8 illustrates the approach using an example where 128 elements are selected from a 4096-dimensional activation vector ($d_{in} = 4096$, $k = 128$). As shown in Figure 8(a), instead of a single global selection, DecDEC partitions the input into four contiguous 1024-dimensional chunks and performs a local Top- k_{chunk} selection within each chunk. In this example, $k_{chunk} = k/4 = 32$. The locally selected elements are then concatenated to form the final result. Although this introduces approximation, it significantly reduces latency by avoiding global synchronization—each local selection is handled independently by a thread block. A larger chunk size can improve efficiency but may increase the approximation error; we set the chunk size to 1024 to balance this trade-off effectively.

For each local selection, DecDEC uses a variant of the bucket-based Top-K algorithm [2]. Figure 8(b) illustrates this process. ❶ First, the 1024 elements in a chunk are scattered into buckets based on their magnitudes, with bucket boundaries ($b_0^k, b_1^k, \dots, b_{30}^k$). The number of buckets is set to 32, matching the number of threads in a warp, allowing for efficient thread-level parallelism so that each thread processes a different bucket. ❷ Next, elements are gathered, starting from bucket 0, until the total count reaches k_{chunk} . ❸ If the

number of elements in the current bucket exceeds the remaining spots for k_{chunk} , as in the case of bucket 9 in Figure 8(b), random selection is used to fill the remaining spots. This random selection adds an additional layer of approximation but significantly reduces latency by avoiding exact sorting.

Determining proper bucket boundaries is crucial to minimize the approximation error introduced by random selection (❸ in Figure 8(b)). DecDEC profiles the distribution of activation values using a small calibration set and aims to set boundary values that balance Top-K accuracy with the ability to handle a broader range of values. Placing finer-grained buckets around the expected k -th largest value generally improves accuracy, but limits the system's ability to handle out-of-distribution values. To address this, DecDEC uses an offline analysis to determine two key boundaries, b_0^k and b_{15}^k , from which the other boundaries are inferred (as shown in Figure 9). Let the distribution of activation values from the calibration set be $\mathbf{X} \in \mathbb{R}^{N \times d_{in}}$, where N is the size of the set. The boundary b_{15}^k is set to the maximum of the k -th largest value across all vectors in $\text{abs}(\mathbf{X})$. The interval between 0 and b_{15}^k is uniformly divided into 16 buckets ($b_{15}^k, b_{16}^k, \dots, b_{30}^k$), focusing on the range where the k -th largest value is most likely to occur. To handle out-of-distribution cases which can cause significant degradation in selection precision, DecDEC assigns an additional 16 buckets for values beyond b_{15}^k . Specifically, b_0^k is set to the maximum value in all $\text{abs}(\mathbf{X})$, and the range between b_0^k and b_{15}^k is also uniformly divided to form the remaining 16 buckets.

Kernel Fusion. DecDEC extensively fuses all dynamic error compensation operations into a single kernel. Figure 10 visualizes the execution flow of the fused kernel, using an example where two thread blocks process a weight matrix of size 4096×6144 . In this example, only one channel is selected per each of the four chunks (i.e., $k_{chunk} = 1$, $k = 4$). Initially, each thread block sequentially processes two chunks, selecting a total of two channels (Step ❶ in Figure 6(b)). The indices for the selected channels (sc_indices) as well as the corresponding activation values ($\mathbf{x}[\text{sc_indices}]$) are stored in GPU memory. Thread blocks are then synchronized using the grid-wide synchronization feature of the cooperative group [43]. This synchronization is required because each thread block

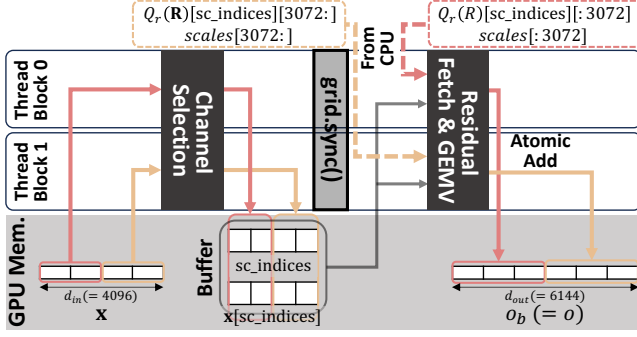


Figure 10: Fused kernel for dynamic error compensation.

processes a segment of all selected channels—not just a disjoint subset—when fetching quantized residuals from the CPU and performing GEMV (Steps ② and ③ in Figure 6(b)). For example, thread block 0 processes $Q_r(\mathbf{R})[\text{sc_indices}][:3072]$ as opposed to $Q_r(\mathbf{R})[\text{sc_indices}[:2]][:]$. The thread block-level synchronization allows all thread blocks to have access to the complete set of `sc_indices` and `x[sc_indices]`. This partitioning scheme allows for efficient reduction in the residual GEMV without requiring extensive global synchronization. The results from the residual GEMV are directly added to the result of the base GEMV ($\mathbf{o}_b$) using atomic primitives, yielding the final output ($\mathbf{o}$) (Step ④ in Figure 6(b)).

GPU Memory Overhead. The buffer for `sc_indices` and `x[sc_indices]` in the fused kernel is the only additional GPU memory usage of DecDEC. Zero-copy from the CPU does not consume GPU memory. The bucket boundary values are not stored on the GPU; instead, only b_0^k and b_{15}^k are passed to the kernel as arguments. A single buffer can be reused for all linear layers if it is sized in accordance with the largest k . In the extreme case of fetching 10% of the channels across all layers in Llama-3-8B, the maximum k would be 1433, for the down projection layer. This calls for an 8.6 KB buffer ($1433 \times (4 + 2)$), which is less than 0.0003% of the model size, assuming 3-bit precision—essentially a negligible overhead.

4.4 Parameter Tuner

Necessity of Parameter Tuner. Effective use of DecDEC requires careful tuning two key parameters:

- n_{tb} : Specifies the number of thread blocks used for dynamic error compensation. Since DecDEC runs in parallel with the base GEMV, allocating too many thread blocks to dynamic error compensation can slow down the base GEMV computation, while allocating too few may underutilize PCIe bandwidth, as zero-copy transfers require GPU cores to issue memory requests.
- k_{chunk} : Specifies how many channels are compensated per chunk (1024 channels). A larger k_{chunk} improves model quality but may increase inference latency.

Choosing n_{tb} and k_{chunk} is challenging due to the large design space. The n_{tb} value for each type of linear layer (n_{tb}^{qkv} , n_{tb}^o , n_{tb}^{gu} , n_{tb}^d) has multiple viable candidates, constrained by the dimensions of the corresponding weight matrices. For example, in Llama-3-8B, there are 9 possible candidates for n_{tb}^{qkv} (1, 2, 3, 4, 5, 6, 8, 12, 24), while other layers also have multiple options. Selecting k_{chunk} presents a greater challenge due to its broader range of possibilities. The k_{chunk} value for each type of linear layer (k_{chunk}^{qkv} , k_{chunk}^o , k_{chunk}^{gu} , k_{chunk}^d) can be any integer less than a maximum determined by the available shared memory. Although platform-dependent, this upper bound is large (e.g., 367 with 48 KB per-block shared memory), leading to an expansive search space. This creates a combinatorial explosion of configuration options. Technical details on how to identify candidate values of k_{chunk} and n_{tb} for a given model and platform are presented at the end of this section.

To address this issue, we provide a DecDEC tuner that suggests n_{tb} and k_{chunk} values based on a target slowdown rate. The tuner maximizes k_{chunk} while keeping the total execution time—including base GEMV and dynamic error compensation across all linear layers—within the specified slowdown relative to the baseline (i.e., without compensation). This tuning is a one-time process for a given model-device pair.

Parameter Tuning Process. Figure 11(a) illustrates the tuning process, which consists of two phases: Phase 1 determines the n_{tb} values, and Phase 2 determines the k_{chunk} values.

In Phase 1, the tuner simplifies the search for n_{tb} values for each layer by replacing it with the search for a single metaparameter, n_{tb}^{max} . n_{tb}^{max} represents the upper limit on the number of thread blocks to use for dynamic error compensation. Each layer's n_{tb} is then set to the largest candidate below n_{tb}^{max} . Determining n_{tb}^{max} involves testing values up to half of the total SM count, to reduce the search space. In the example shown in Figure 11, the GPU has 56 SMs, so values up to 28 are tested. For each tested n_{tb}^{max} , a coarse-grained k_{chunk} search checks how many uniform increments to k_{chunk} can be applied across all layers without exceeding the target slowdown rate. Figure 11(b) shows an example of a coarse-grained k_{chunk} search for $n_{tb}^{max} = 24$ (i.e., $(n_{tb}^{qkv}, n_{tb}^o, n_{tb}^{gu}, n_{tb}^d) = (24, 16, 23, 14)$), yielding 19 valid steps. If no steps can be made for any n_{tb}^{max} value, the tuner fixes k_{chunk} to 0 for the layer with the smallest weight matrix and repeats the process, as smaller matrices are often most sensitive to increases in k_{chunk} .

After Phase 1, the tuner selects the n_{tb}^{max} with the most steps and proceeds to a fine-grained k_{chunk} search in Phase 2. Figure 11(c) shows the fine-grained k search for the selected $n_{tb}^{max} = 24$. In this phase, not all k_{chunk} values may increase together. At each step, the tuner increments k_{chunk} for as many layers as possible, prioritizing those with smaller increases in execution time. For example, in Step 1, k_{chunk}^{gu} is incremented first, followed by k_{chunk}^d . Step 1 stops at this point, as further increases to k_{chunk}^{qkv} and k_{chunk}^o would exceed the target; thus

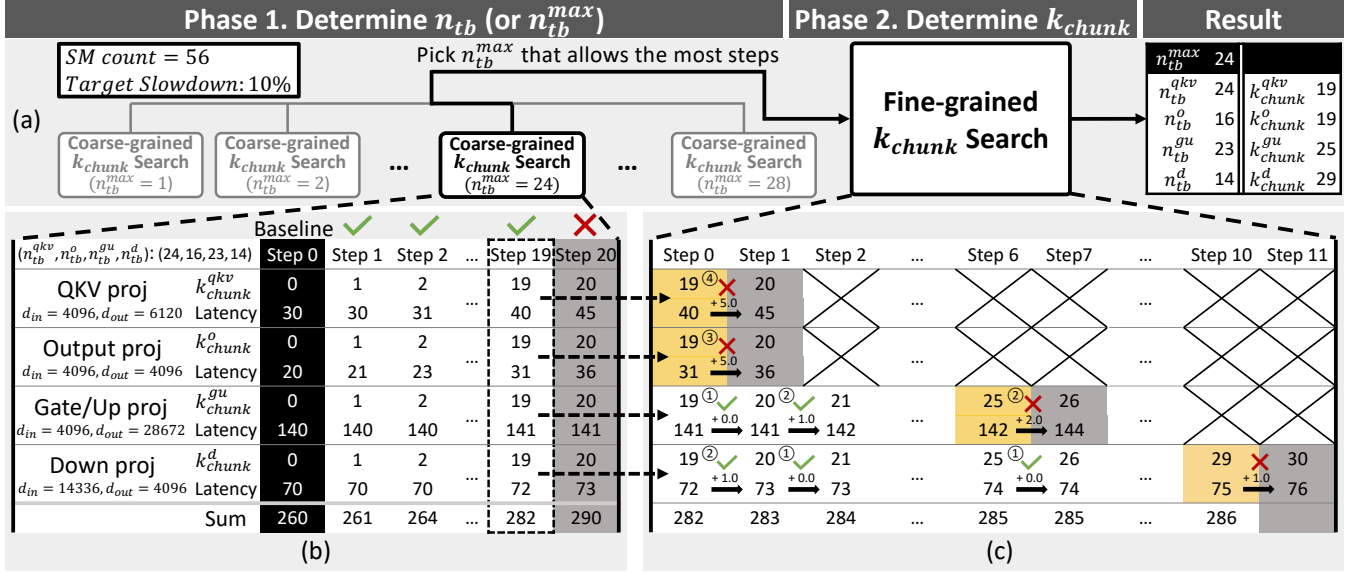


Figure 11: Parameter tuning process for DecDEC, assuming a total of 56 SMs and a target slowdown rate of 10%.

their final values are set (i.e., $k_{chunk}^{qkv}, k_{chunk}^o = 19$). This process repeats until no further increments can be made for any layer. **Technical Details: n_{tb} and k_{chunk} Candidates.** The n_{tb} values considered during parameter tuning are those that have a meaningful impact on at least one of the two parts of the kernel execution: the approximate Top-K selection and the residual fetching. In the approximate Top-K selection part, the minimum processing granularity per thread block is one chunk. Therefore, increasing n_{tb} beyond the number of chunks does not have additional effect on performance. Thus, the set of n_{tb} values relevant to this part is:

$$A = \left\{ n \mid 1 \leq n \leq \frac{d_{in}}{1024} \right\}.$$

For residual fetching part, 4-bit residuals are transferred over PCIe in coalesced segments of 256 values (128 bytes), resulting in a total of $s = d_{out}/256$ segments. These segments are distributed across n_{tb} thread blocks, with each block processing $\lceil s/n_{tb} \rceil$ segments. If multiple n_{tb} values result in the same number of segments per block ($\lceil s/n_{tb} \rceil$), only the smallest such value is considered and the rest are redundant. Excluding these cases, the candidate set relevant to this part is:

$$B = \left\{ n \mid 1 \leq n \leq s, \left\lceil \frac{s}{n} \right\rceil = n \right\}.$$

The final candidate set for n_{tb} is the union of the two (i.e., $N = A \cup B$).

Meanwhile, k_{chunk} is bounded by the shared memory limit. During the approximate Top-K selection part of the kernel, shared memory usage increases with the value of k_{chunk} . Specifically, the shared memory usage for this part is:

$$128 + 128 \times k_{chunk} + 2 \times 1024 \text{ bytes.}$$

GPU Name	Memory Size	Memory BW	# SM	PCIe BW	R_{bw}
Desktop					
RTX 4090	24 GB	1,008 GB/s	128	32 GB/s	32
RTX 4080S	16 GB	736 GB/s	80	32 GB/s	23
RTX 4070S	12 GB	504 GB/s	56	32 GB/s	16
Laptop					
RTX 4070M	8 GB	256 GB/s	36	16 GB/s	16
RTX 4050M	6 GB	192 GB/s	20	16 GB/s	12

Table 1: GPU specifications.

Here, 128 bytes are used for integer counters that track the number of elements that fall into each of the 32 buckets; $128 \times k_{chunk}$ bytes are used to temporarily store the indices of elements assigned to each bucket; and 2×1024 bytes account for the input activation values in the chunk. The total must remain below the per-block shared memory limit (e.g., 49,152 bytes), which constrains the maximum allowable k_{chunk} value.

5 Evaluation

5.1 GPU Kernel Benchmarks

Methodology. In this section, we evaluate the DecDEC GPU kernel on three consumer-grade GPUs: two desktop GPUs, RTX 4070 Super (RTX 4070S) and RTX 4090, and one laptop GPU, RTX 4050 Mobile (RTX 4050M), with their specifications detailed in Table 1. Here, R_{bw} denotes the ratio of memory bandwidth to CPU-to-GPU (PCIe) bandwidth. The evaluation considers GEMV operations of output, gate/up and down projection layers in Llama-3-8B-Instruct [16], assuming 3-bit bitwidth. For the base GEMV, we use the LUTGEMM kernel, [23], a state-of-the-art GEMV kernel for uniform quantization. Kernel times are measured using NVIDIA Nsight Systems.

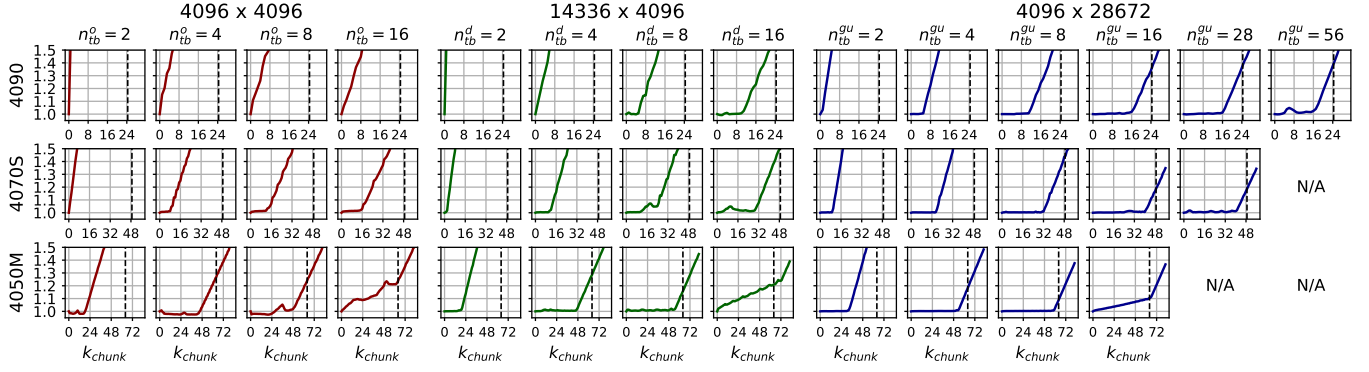


Figure 12: Execution time of base GEMV + DecDEC with varying k_{chunk} and n_{tb} , normalized to base GEMV execution time.

Expected Behavior. The execution time of the DecDEC kernel is expected to follow a piecewise linear function of k_{chunk} with two distinct segments. In the first segment, corresponding to small k_{chunk} values, the latency remains nearly constant at the base GEMV time, as the compensation operations are fully hidden under the GEMV execution. In the second segment, once k_{chunk} exceeds a certain threshold—the knee point—the execution time linearly increases with k_{chunk} , indicating that the compensation operations are no longer fully hidden. Theoretically, the knee point is expected to occur when $k_{chunk} = 1024 \times 1/R_{bw} \times 3/4$, representing the maximum amount of data transfer that can overlap with the base GEMV. Here, the base GEMV time is approximated by dividing the weight matrix size by the GPU memory bandwidth, and PCIe bandwidth is assumed to be fully utilized. The factor $3/4$ accounts for the 3-bit quantization; for other bitwidths, this factor should be adjusted accordingly (e.g., $4/4 = 1$ for 4-bit quantization).

Results. Figure 12 shows the execution time of DecDEC kernel (base GEMV + dynamic error compensation), normalized to the standalone execution time of the base GEMV, across varying n_{tb} and k_{chunk} . Each subfigure also includes a vertical dotted line to mark the theoretical knee point derived from the analytical model (i.e., $1024 \times 1/R_{bw} \times 3/4$). All cases exhibit the expected two-segment piecewise linear behavior when n_{tb} is properly set, except for the 4096×4096 case on RTX 4090, the fastest GPU evaluated. In this case, the base GEMV execution time is so short that even a small k_{chunk} incurs overhead.

From the other cases, three key observations can be made. First, a lower R_{bw} —indicating lower memory bandwidth and higher PCIe bandwidth—shifts the knee point to the right. Thus, RTX 4050M, with the lowest R_{bw} , supports the largest k_{chunk} before the knee point, while RTX 4090, with the highest ratio, supports the smallest. This trend is consistent with the ordering predicted by theoretical knee points. Second, the knee point is highly sensitive to n_{tb} , highlighting the importance of careful tuning. Generally, higher n_{tb} values such as 8 or 16 delay the knee point. In contrast, small n_{tb} values (e.g., 2) cause

the knee point to appear too early or disappear, leading to sub-optimal performance. However, increasing n_{tb} can sometimes worsen results by slowing down the base GEMV, as discussed in Section 4.4. This effect is particularly noticeable on GPUs with fewer SMs, such as the 4050M. For instance, on the 4050M, $n_{tb} = 8$ yields the best results, whereas increasing n_{tb} to 16 leads to worse performance. Third, larger weight sizes enable higher k_{chunk} with minimal latency overhead, due to increased time slack. With sufficiently large weight sizes (e.g., 4096×28672) and a properly tuned n_{tb} , the actual knee point approaches the theoretical value. For example, on an RTX 4050M with a 4096×28672 matrix and $n_{tb} = 8$, the observed knee point is around $k_{chunk} = 60$, compared to a theoretical prediction of 64.

5.2 Impact on Model Quality

Methodology. We demonstrate DecDEC’s quality improvements on 3-bit, 3.5-bit, and 4-bit versions of two instruction-tuned LLMs: Llama-3-8B-Instruct [16] (hereafter referred to as Llama-3) and Phi-3-medium-4k-instruct (14B model, hereafter referred to as Phi-3) [40]. For the 3.5-bit version, we adopt a block-wise bitwidth allocation, applying 3-bit quantization to half of the decoder blocks and 4-bit quantization to the remaining blocks. This follows a KL divergence-based sensitivity metric from prior work [8].

As base quantization methods, we choose two state-of-the-art LLM quantization methods: AWQ [36] and SqueezeLLM [31]. AWQ is a *uniform* quantization method that mitigates quantization errors by applying per-channel scaling to protect salient channels, which are identified through an offline analysis on a calibration dataset. SqueezeLLM employs a clustering-based *non-uniform* quantization method that considers the sensitivity of each weight.

We evaluate DecDEC-augmented models across varying k_{chunk} , the number of channels compensated per chunk (1024 channels). We uniformly set k_{chunk} to 8, 16, 32, 64 and 128 for all layers. Based on the results in Section 5.1, values up to 64 fall within the practical range—where latency overhead from

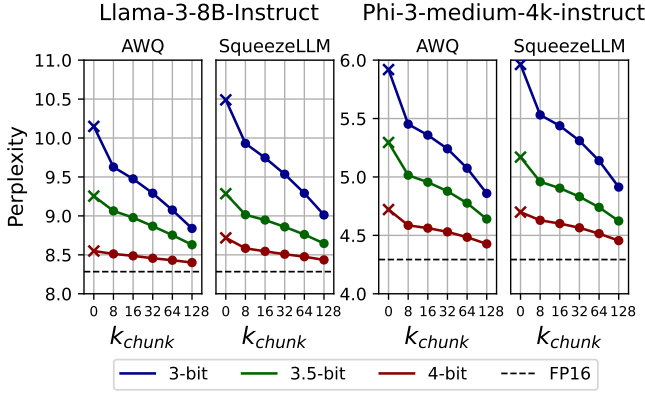


Figure 13: Perplexity on WikiText. The x markers correspond to baselines without DecDEC ($k_{\text{chunk}} = 0$). Lower is better.

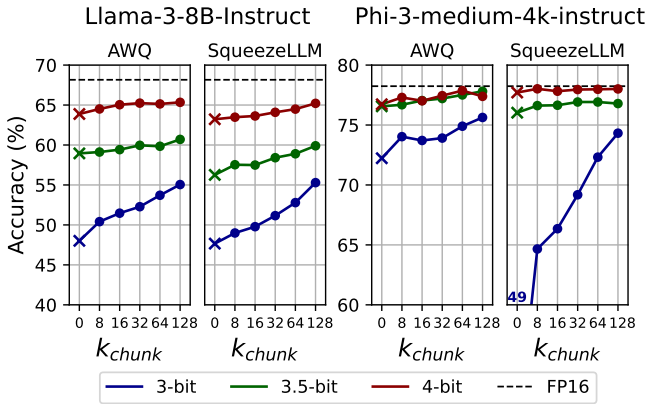


Figure 14: Accuracy on BBH. The x markers correspond to baselines without DecDEC ($k_{\text{chunk}} = 0$). Higher is better.

dynamic error compensation may remain low depending on the platforms—while $k_{\text{chunk}} = 128$ is included for completeness in evaluating upper-bound behavior.

Benchmarks. Following previous literature [9, 12, 31, 36], we use perplexity on WikiText [39] as the primary metric, as it reliably reflects quantized LLM quality [13, 36]. Additionally, we use BIG-Bench Hard (BBH) [54], a collection of 23 challenging tasks in BIG-Bench [52], to assess the models’ capabilities in problem solving. Chain-of-Thought (CoT) is enabled for all BBH evaluations [62]. Lastly, we evaluate multi-turn conversation using MT-Bench [68], where a strong LLM judges 80 responses, scoring each from 0 to 10. We use GPT-4o as the judge and report the average of three runs.

Perplexity on WikiText. Figure 13 shows the perplexity results. Across all cases, a clear trend is observed: perplexity consistently decreases (i.e., model quality improves) as k_{chunk} increases. In particular, for 3-bit models, significant improvements occur even at $k_{\text{chunk}} = 8$. AWQ’s perplexity decreases from 10.15 to 9.63 for Llama-3 and from 5.96 to 5.53 for Phi-3, while SqueezeLLM’s perplexity drops from 10.49 to

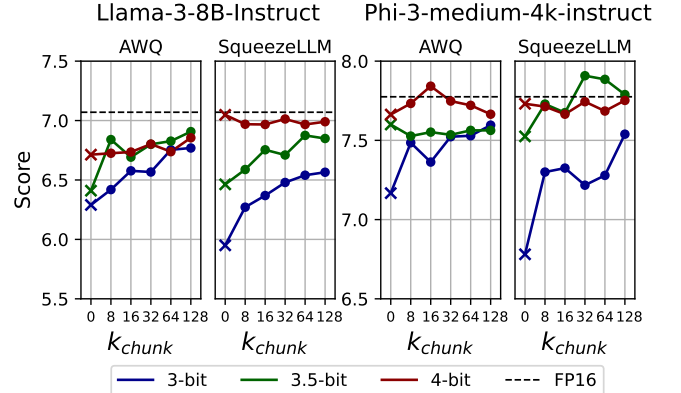


Figure 15: MT-Bench scores. The x markers correspond to baselines without DecDEC ($k_{\text{chunk}} = 0$). Higher is better.

9.93 for Llama-3 and from 5.92 to 5.45 for Phi-3. Meanwhile, the impact on 4-bit models is relatively less pronounced. This is expected, as 4-bit models are already close to full-precision, leaving less room for improvement. The 3.5-bit models follow an intermediate trend.

BBH and MT-Bench. Results on BBH (Figure 14) follow the same trends as the perplexity results. The MT-Bench results (Figure 15), on the other hand, demonstrate some different patterns. In cases where vanilla quantized models without DecDEC ($k = 0$) already achieve scores very close to those of the FP16 model—such as all 4-bit cases and the AWQ 3.5-bit model of Phi-3—the scores remain unchanged, oscillating around the baseline score. For the remaining cases, DecDEC significantly improves scores even with a small k_{chunk} (e.g., 8), like in other benchmarks; further increases in k_{chunk} , however, do not always yield noticeable improvements. These patterns may be attributed to the coarse-grained rubric of this benchmark, which assigns integer scores ranging from 0 to 10 for each task. This coarse-grained scoring may miss subtle improvements when the potential gain is small.

Effectiveness of DecDEC’s Channel Selection. Figure 16 compares DecDEC with three variants using different channel selection mechanisms. To assess the benefits of dynamic selection, we include *Random* (blue line), which selects channels randomly, and *Static* (yellow line), which statically select channels by Hessian-based ranking with a calibration set (using exact sorting) following prior work [33]. To isolate the effect of DecDEC’s Top-K approximation, we include *Exact* (red line), which uses true Top-K channels. Alongside perplexity results, we also report the average recall rate of Static and DecDEC relative to Exact.

While Static improves over Random, it underperforms DecDEC. DecDEC achieves lower perplexity than Static while using 4× or even 8× fewer channels (e.g., $k_{\text{chunk}} = 32$ or 16 vs. 128), highlighting the effectiveness of incorporating input-awareness. Meanwhile, the perplexity gap between DecDEC and Exact is minimal, with almost overlap

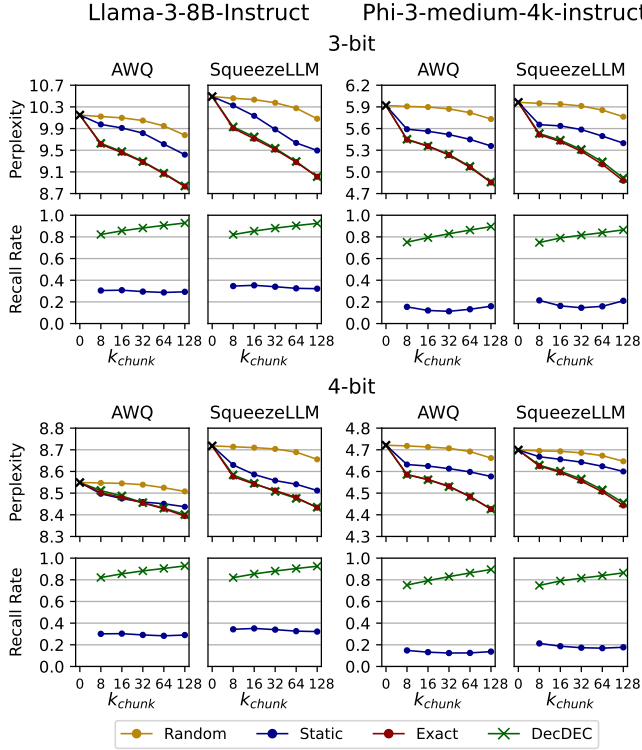


Figure 16: Comparison against random, static, and exact channel selection. Perplexity is shown on the top rows (lower is better), and recall is shown on the bottom rows (higher is better). The x markers in black correspond to baselines without DecDEC ($k_{chunk} = 0$).

ping curves. These results are explained by the recall rates: while DecDEC achieves around 80% recall relative to Exact, the recall of Static falls significantly short, at around 30% or below.

Impact of Residual Bitwidth. Table 2 presents the impact of residual bitwidth selection. In addition to the default 4-bit setting, we evaluate 2-bit, 8-bit, and full-precision (FP16) residuals with varying k_{chunk} for 3-bit models by perplexity on WikiText. Cells with the same color indicate cases where the total data transfer via PCIe is approximately equivalent. For example, $k_{chunk} = 8$ with 4-bit residuals requires a similar data transfer amount as the following combinations: $k_{chunk} = 16$ at 2-bit, $k_{chunk} = 4$ at 8-bit, and $k_{chunk} = 2$ at FP16. The best results within each color group are highlighted. Across all cases, a residual bitwidth of 4 either achieves the best or comes very close to it, supporting our default setting.

5.3 End-to-End Evaluation

Methodology. In this section, we present case studies demonstrating how DecDEC advances model quality while minimizing latency increases across three desktop GPUs—RTX 4090, 4080 Super (4080S), and 4070 Super (4070S)—as well as two

k_{chunk}	Llama-3-8B-Instruct					Phi-3-medium-4k-instruct				
	AWQ 3-bit		SqueezeLLM 3-bit			AWQ 3-bit		SqueezeLLM 3-bit		
	2-bit	4-bit	8-bit	FP16		2-bit	4-bit	8-bit	FP16	
2			9.83		10.10			5.56		5.63
4		9.72	9.72		9.97		5.51	5.51		5.55
8	9.63	9.58	9.58		9.93	9.79	9.78	5.45	5.45	5.53
16	9.63	9.47	9.43	9.43	9.95	9.75	9.59	5.47	5.36	5.51
32	9.49	9.29	9.25	9.25	9.80	9.54	9.36	5.36	5.24	5.41
64	9.31	9.07	9.03		9.59	9.29	9.11	5.22	5.08	5.28
128	9.12	8.84			9.36	9.01		5.04	4.86	5.10
256	8.94				9.14			4.83		4.91

Table 2: Impact of residual bitwidth. Lower is better.

laptop GPUs—RTX 4070 Mobile (4070M), and 4050 Mobile (4050M). The specifications for these GPUs are listed in Table 1. We run the tuner for DecDEC with four target slowdown rates (2.5%, 5%, 10%, 20%), and evaluate perplexity on WikiText as well as end-to-end inference latency with the resulting configurations. For the base GEMV kernel, we use LUTGEMM [23] for AWQ and Any-Precision LLM [45] for SqueezeLLM—both state-of-the-art kernels for uniform and non-uniform quantization, respectively. We integrate DecDEC into a PyTorch-based inference pipeline optimized with the torch.compile feature [5] and measure the average time taken per token generation over 1024 tokens.

For the 3.5-bit models, we do not run the tuner separately. Instead, we construct the configuration by combining the tuning results with the 3-bit and 4-bit models. Specifically, we use the configuration from tuning with the 3-bit model for layers quantized to 3 bits, and the configuration from tuning with the 4-bit model for layers quantized to 4 bits—both assuming the same target slowdown rate.

Results. Table 3 lists the configurations obtained from the tuner along with the actual end-to-end slowdowns compared to the baseline. We report only the results for 3-bit as similar trends are observed for the 3.5-bit and 4-bit configurations. In all cases, the actual slowdown is below the target rate. This is expected as the tuner configures parameters conservatively. The tuner targets only the kernel times of linear operations, while other operations outside the linear layers (e.g., attention, normalization) also contribute to overall inference time. Consistent with the observations in Section 5.1, in general, selected k values are higher for GPUs with greater PCIe-to-GPU memory bandwidth ratio (4050M > 4070M $\approx$ 4070S > 4080S > 4090).

Figure 17 shows the trends of end-to-end inference latency versus perplexity. All Phi-3 cases, as well as AWQ 3.5-bit, 4-bit, and SqueezeLLM 4-bit cases of Llama-3 face out-of-memory issues on the 4050M, and are thus excluded. Similarly, the AWQ 4-bit case of Phi-3 for 4070M is also excluded. In each line in Figure 17, the x marker represents the baseline, while subsequent markers show the results for DecDEC on the four target slowdown rates, in increasing order.

For all cases, DecDEC demonstrates promising Pareto-optimal trade-offs between model quality and inference latency. The results for target slowdown rates of 2.5% and 5% are particularly impressive, while 10%–20% show diminish-

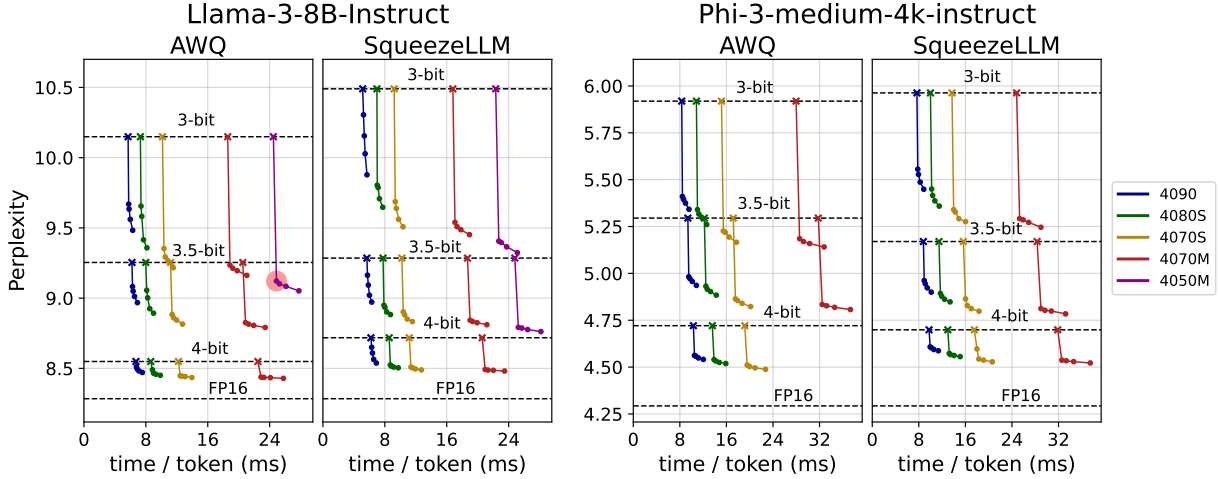


Figure 17: Perplexity against time per token on various NVIDIA GPUs. The x markers indicate baseline values, where DecDEC is not applied ($k_{\text{chunk}} = 0$). Subsequent markers show DecDEC results on target slowdown rates 2.5%, 5%, 10%, and 20%.

		Llama-3-8B-Instruct				Phi-3-medium-4k-instruct			
		AWQ 3-bit		SqueezeLLM 3-bit		AWQ 3-bit		SqueezeLLM 3-bit	
GPU	Target	Tuner Results	Slowdown	Tuner Results	Slowdown	Tuner Results	Slowdown	Tuner Results	Slowdown
4090	2.5%	24 / (4, 4, 8, 9)	1.3%	56 / (2, 0, 2, 2)	2.0%	20 / (8, 6, 13, 13)	1.5%	47 / (5, 2, 7, 10)	1.8%
	5%	24 / (5, 7, 9, 10)	2.2%	56 / (1, 1, 2, 6)	4.0%	47 / (10, 11, 13, 13)	3.8%	35 / (6, 6, 9, 10)	3.5%
	10%	24 / (10, 11, 11, 11)	4.9%	38 / (5, 4, 5, 7)	6.1%	35 / (15, 15, 15, 14)	7.8%	47 / (10, 11, 13, 11)	7.3%
	20%	24 / (15, 15, 16, 15)	10.4%	28 / (10, 10, 10, 10)	10.8%	20 / (18, 19, 18, 18)	15.0%	35 / (16, 16, 15, 14)	15.3%
4080S	2.5%	6 / (9, 5, 9, 6)	1.1%	24 / (10, 11, 14, 16)	0.6%	30 / (18, 17, 18, 20)	1.7%	30 / (14, 14, 15, 16)	2.0%
	5%	16 / (10, 10, 11, 9)	2.5%	24 / (14, 14, 14, 14)	2.2%	30 / (21, 21, 23, 21)	3.9%	30 / (17, 17, 20, 19)	3.7%
	10%	24 / (18, 20, 23, 18)	5.5%	24 / (18, 19, 18, 17)	4.7%	35 / (24, 25, 24, 24)	8.3%	35 / (22, 22, 23, 21)	7.6%
	20%	24 / (26, 27, 25, 24)	11.5%	38 / (23, 24, 22, 23)	10.8%	35 / (30, 32, 28, 29)	16.2%	35 / (27, 27, 25, 25)	15.1%
4070S	2.5%	28 / (25, 26, 26, 26)	2.0%	19 / (16, 15, 24, 21)	1.5%	20 / (31, 31, 37, 32)	2.1%	20 / (25, 26, 30, 28)	2.1%
	5%	24 / (31, 31, 35, 29)	3.7%	24 / (21, 22, 24, 24)	3.0%	20 / (35, 35, 37, 34)	4.2%	20 / (30, 30, 30, 29)	4.2%
	10%	24 / (35, 35, 36, 34)	7.1%	24 / (28, 30, 30, 27)	6.2%	24 / (39, 40, 41, 38)	8.5%	24 / (34, 35, 35, 33)	8.6%
	20%	28 / (44, 44, 41, 42)	13.9%	24 / (35, 36, 34, 35)	12.4%	20 / (45, 46, 44, 45)	17.0%	20 / (40, 41, 38, 39)	16.9%
4070M	2.5%	16 / (38, 39, 39, 40)	1.7%	12 / (28, 28, 35, 29)	1.6%	18 / (41, 41, 41, 42)	2.1%	17 / (33, 35, 35, 34)	2.1%
	5%	16 / (42, 43, 42, 42)	3.4%	16 / (33, 34, 36, 33)	3.1%	18 / (44, 46, 43, 44)	4.4%	18 / (37, 38, 37, 37)	4.3%
	10%	16 / (46, 46, 44, 45)	6.6%	16 / (37, 38, 37, 37)	6.1%	18 / (46, 48, 46, 46)	8.3%	18 / (40, 41, 39, 39)	8.5%
	20%	16 / (50, 50, 49, 50)	13.4%	16 / (42, 42, 41, 41)	12.7%	18 / (53, 53, 50, 51)	17.2%	18 / (44, 44, 43, 43)	16.7%
4050M	2.5%	8 / (55, 56, 58, 55)	1.7%	7 / (45, 45, 50, 44)	1.7%	OOM	-	OOM	-
	5%	8 / (59, 59, 59, 58)	3.3%	9 / (48, 48, 48, 48)	3.3%	OOM	-	OOM	-
	10%	10 / (62, 62, 62, 62)	6.6%	10 / (52, 53, 52, 52)	6.3%	OOM	-	OOM	-
	20%	10 / (70, 70, 68, 68)	13.5%	10 / (59, 60, 57, 58)	12.6%	OOM	-	OOM	-

Table 3: Tuner results ($n_{\text{tib}}^{\text{max}} / (k_{\text{chunk}}^{\text{kv}}, k_{\text{chunk}}^{\text{o}}, k_{\text{chunk}}^{\text{gu}}, k_{\text{chunk}}^{\text{d}})$) for four target slowdown rates (2.5%, 5%, 10%, 20%) and corresponding actual slowdown rates for 3-bit Llama-3 and Phi-3. Phi-3 is out of memory (OOM) on the 4050M GPU.

ing returns. On platforms with high PCIe-to-GPU memory bandwidth ratios, such as the 4070S, 4070M, and 4050M, DecDEC at 2.5% slowdown on 3-bit models sometimes outperforms 3.5-bit baselines. In these cases, DecDEC yields *Pareto-dominant* solutions by excelling in model quality, latency, and memory. Examples include AWQ Llama-3 (4070M, 4050M) and AWQ Phi-3 (4070S, 4070M). A particularly noteworthy case is the AWQ 3-bit Llama-3 on 4050M, where DecDEC reduces perplexity from 10.15 to 9.12 with only a 1.7% latency slowdown (highlighted by a red circle). This outperforms the 3.5-bit baseline, which is infeasible on 4050M

without DecDEC due to memory limits. This case highlights DecDEC’s effectiveness in pushing the boundaries of quantized LLMs within memory capacity constraints.

5.4 Evaluation across GPU Generations

We evaluate the robustness of DecDEC across GPU generations, as both GPU memory bandwidth and PCIe bandwidth, two key factors influencing its effectiveness, may increase with newer architectures. To this end, we use three consumer-grade GPUs from the same product class across different

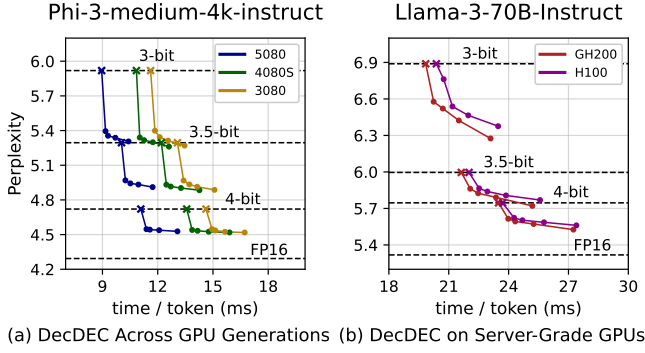


Figure 18: Perplexity vs. time per token: (a) across different GPU generations and (b) on server-grade GPUs.

	Memory Bandwidth	PCIe Bandwidth	R_{bw}
RTX 5080	960 GB/s	64 GB/s	15
RTX 4080S	736 GB/s	32 GB/s	23
RTX 3080	760 GB/s	32 GB/s	24

Table 4: 80-class GPU specifications across generations.

generations—RTX 3080, 4080S, and 5080—whose specifications are listed in Table 4. The R_{bw} values (lower is better) remain essentially unchanged from the 3080 to the 4080S and even decrease from the 4080S to the 5080. Figure 18(a) plots the perplexity of AWQ-quantized Phi-3 with DecDEC augmentation against end-to-end latency on these GPUs, using the methodology described in Section 5.3. The improvements delivered by DecDEC are comparable across all three cards, confirming its robustness across generations.

5.5 Applicability to Server-Grade GPUs

Although DecDEC primarily targets single-batch inference on client (edge) devices, this section evaluates its effectiveness on server-grade GPUs. We measure perplexity versus end-to-end latency using an AWQ-quantized Llama-3-70B-Instruct model augmented with DecDEC on the H100 SMX5 and GH200, following the methodology described in Section 5.3. Both GPUs provide 3.36 TB/s of memory bandwidth; however, the GH200’s 450 GB/s NVLink-C2C far exceeds the H100’s 64 GB/s PCIe, yielding a much lower R_{bw} .

Figure 18(b) shows the results. DecDEC improves perplexity on both GPUs with minimal latency overhead. However, the GH200’s advantage is smaller than the R_{bw} gap suggests. DecDEC assumes a DRAM-bound GEMV, where real-locating SMs for error compensation do not impact GEMV latency. On server GPUs, however, quantized GEMV (e.g., LUTGEMM [23]) is L1-bound, not DRAM-bound. Since L1 throughput scales with active SMs, reallocating SMs increases GEMV latency, limiting DecDEC’s benefit despite low R_{bw} . Enhancing quantized GEMV kernels for server-grade GPUs by mitigating L1 bottlenecks could unlock further gains.

6 Related Work

GPU Implementations for Weight-Only Quantization. Numerous studies have proposed efficient GPU implementations tailored for weight-only quantization of LLMs. LUT-GEMM [23] replaces the expensive dequantization with simple LUT operations. Marlin [20] supports FP16-INT4 GEMM across various batch sizes, and Quant-LLM [63] introduces a GPU kernel that efficiently handles non-power-of-2 bitwidths (e.g., 6-bit). FLUTE [24] provides a specialized kernel for non-uniform quantization. Any-Precision LLM [45] suggests a memory-efficient kernel for adaptive bitwidth selection. These implementations can seamlessly integrate with DecDEC, benefiting from its dynamic error compensation.

LLM Inference with External Memory. Several works address GPU memory limitations in LLM inference by leveraging external memories like CPU memory or disk. DeepSpeed-Inference [4] and FlexGen [50] focus on throughput-oriented, out-of-core LLM inference. Pre-gated MoE [29] retrieves the parameters of only the activated experts in MoE models from CPU memory. LLM-in-a-Flash [3] introduces a flash memory-based inference system exploiting activation sparsity. InfiniGen [35] offloads the key-value cache to CPU memory. PowerInfer [51] proposes a GPU-CPU hybrid inference engine that leverages activation sparsity in ReLU-based models. Though these approaches share the same goal as DecDEC in aiming to extend GPU memory, they address distinct scenarios with different challenges and opportunities.

LLM Compression Methods. In addition to quantization, pruning can improve the efficiency of LLM inference [18, 38, 53]. Other techniques, including knowledge distillation [22, 26] and low-rank decomposition [48, 67], offer additional ways to compress LLMs. These methods are orthogonal to quantization and, consequently, to DecDEC.

7 Conclusion

We propose DecDEC, an inference scheme for low-bit LLMs that improves model quality by correcting quantization errors through selective retrieval of residuals stored in CPU memory. By focusing on dynamically identified salient channels at each decoding step, DecDEC maximizes error compensation within a limited transfer volume. DecDEC significantly improves quality of quantized LLMs with minimal overheads.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grants funded by the Korea government (MSIT) (RS-2024-00340008, RS-2024-00405857). Jae W. Lee is the corresponding author.

References

- [1] NVIDIA H100 Tensor Core GPU. <https://resources.nvidia.com/en-us-data-center-overview-mc/en-us-data-center-overview/nvidia-tensor-core-gpu-datasheet>, 2024.
- [2] Tolu Alabi, Jeffrey D. Blanchard, Bradley Gordon, and Russel Steinbach. Fast k -selection algorithms for graphics processing units. *ACM J. Exp. Algorithmics*, 2012.
- [3] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2022.
- [5] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024.
- [6] Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. Quantizable transformers: Removing outliers by helping attention heads do nothing. In *Advances in Neural Information Processing Systems*, 2023.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [8] Yaohui Cai, Zhewei Yao, Zhen Dong, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Zeroq: A novel zero shot quantization framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [9] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems*, 2023.
- [10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2024.
- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [12] Tim Dettmers, Ruslan A. Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alishtarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *The Twelfth International Conference on Learning Representations*, 2024.
- [13] Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [14] Zhen Dong, Zhewei Yao, Daiyaan Arfeen, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. In *Advances in Neural Information Processing Systems*, 2020.
- [15] Zhen Dong, Zhewei Yao, Amir Gholami, Michael Mahoney, and Kurt Keutzer. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *2019 IEEE/CVF International Conference on Computer Vision*, 2019.

- [16] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, and et al Angela Fan. The llama 3 herd of models, 2024.
- [17] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [18] Elias Frantar and Dan Alistarh. Sparsegpt: massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [19] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023.
- [20] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. Marlin: Mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2025.
- [21] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2020.
- [22] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [23] Minsub Kim Sungjae Lee Jeonghoon Kim Beomseok Kwon Se Jung Kwon Byeongwook Kim Youngjoo Lee Gunho Park, Baeseong Park and Dongsoo Lee. Lutgemmm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [24] Han Guo, William Brandon, Radostin Cholakov, Jonathan Ragan-Kelley, Eric P. Xing, and Yoon Kim. Fast matrix multiplications for lookup table-quantized LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- [25] Jung Hwan Heo, Jeonghoon Kim, Beomseok Kwon, Byeongwook Kim, Se Jung Kwon, and Dongsoo Lee. Rethinking channel dimensions to isolate outliers for low-bit weight quantization of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [26] Cheng-Yu Hsieh, Chun-Liang Li, Chih-kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.
- [27] Wei Huang, Haotong Qin, Yangdong Liu, Yawei Li, Xi-anlong Liu, Luca Benini, Michele Magno, and Xiaojuan Qi. Slim-llm: Saliency-driven mixed-precision quantization for large language models. 2024.
- [28] Wei Huang, Xingyu Zheng, Xudong Ma, Haotong Qin, Chengtao Lv, Hong Chen, Jie Luo, Xiaojuan Qi, Xianglong Liu, and Michele Magno. An empirical study of llama3 quantization: From llms to mllms, 2024.
- [29] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture*, 2024.
- [30] Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joon-suk Park, Kang Min Yoo, Se Jung Kwon, and Dongsoo Lee. Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization. In *Advances in Neural Information Processing Systems*, 2023.
- [31] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with page-dattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [33] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. Owq: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [34] Janghwan Lee, Minsoo Kim, Seungcheol Baek, Seok Hwang, Wonyong Sung, and Jungwook Choi. Enhancing computation efficiency in large language models

- through weight and activation quantization. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [35] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
 - [36] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.
 - [37] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. LLM-QAT: Data-free quantization aware training for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
 - [38] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems*, 2023.
 - [39] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
 - [40] Microsoft. Phi-3 technical report: A highly capable language model locally on your phone, 2024.
 - [41] Seung Won Min, Kun Wu, Sitao Huang, Mert Hidayetoğlu, Jinjun Xiong, Eiman Ebrahimi, Deming Chen, and Wen-mei Hwu. Large graph convolutional network training with gpu-oriented data communication architecture. In *Proc. VLDB Endow.*, 2021.
 - [42] NVIDIA. CUDA C++ Best Practices Guide. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>, 2024.
 - [43] NVIDIA. CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024.
 - [44] OpenAI. Gpt-4 technical report, 2024.
 - [45] Yeonhong Park, Jake Hyun, SangLyul Cho, Bonggeun Sim, and Jae W. Lee. Any-precision llm: Low-cost deployment of multiple, different-sized llms. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
 - [46] Carl Pearson, Abdul Dakkak, Sarah Hashash, Cheng Li, I-Hsin Chung, Jinjun Xiong, and Wen-Mei Hwu. Evaluating characteristics of cuda communication primitives on high-bandwidth interconnects. In *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, 2019.
 - [47] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 2020.
 - [48] Varun Srivastava Rajarshi Saha and Mert Pilanci. Matrix Compression via Randomized Low Rank and Low Precision Factorization. In *Advances in Neural Information Processing Systems*, 2023.
 - [49] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang Zhang, Peng Gao, Yu Qiao, and Ping Luo. Omniquant: Omnidirectionally calibrated quantization for large language models. 2023.
 - [50] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: high-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
 - [51] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024.
 - [52] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. 2022.
 - [53] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. 2023.
 - [54] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. 2022.
 - [55] Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.

- [56] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models, 2023.
- [57] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP β : Even better LLM quantization with hadamard incoherence and lattice codebooks. In *Forty-first International Conference on Machine Learning*, 2024.
- [58] turboderp. ExLlamaV2. <https://github.com/turboderp/exllamav2>.
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017.
- [60] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. Efficient large language models: A survey, 2024.
- [61] Zhe Wang, Jie Lin, Xue Geng, Mohamed M. Sabry Aly, and Vijay Chandrasekhar. Rdo-q: Extremely fine-grained channel-wise quantization via rate-distortion optimization. In *Computer Vision – ECCV 2022*, 2022.
- [62] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2024.
- [63] Haojun Xia, Zhen Zheng, Xiaoxia Wu, Shiyang Chen, Zhewei Yao, Stephen Youn, Arash Bakhtiari, Michael Wyatt, Donglin Zhuang, Zhongzhu Zhou, Olatunji Ruwase, Yuxiong He, and Shuaiwen Leon Song. Quant-LLM: Accelerating the serving of large language models via FP6-Centric Algorithm-System Co-Design on modern GPUs. In *2024 USENIX Annual Technical Conference*, 2024.
- [64] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [65] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. In *Advances in Neural Information Processing Systems*, 2022.
- [66] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022.
- [67] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Yan Yan, and Guangyu Sun. Asvd: Activation-aware singular value decomposition for compressing large language models, 2023.
- [68] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2024.
- [69] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models, 2024.